



Concurrency with PHP and Laravel

Huy Nguyen - GHTK Tech



Who am I

- PHP lover, Laravel enthusiast
- Over 7 years experience
- Technical Leader at GHTK Tech Center

My main interests: Software Architecture, Distributed system, High Performance

Love sharing, writing



Concurrency vs Parallel



Concurrency vs Parallel

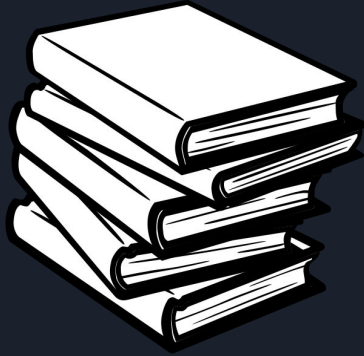
- Parallel chú trọng vào việc xử lý nhiều công việc cùng một lúc
- Concurrency là việc THIẾT KẾ để chạy nhiều công việc cùng một lúc

Concurrency là gì ?





Parallel



Concurrency



Concurrency





Concurrency

Concurrency là cách ta tổ chức phối hợp giữa các tiến trình, sao cho nhận được hiệu quả tốt hơn khi tăng số tiến trình



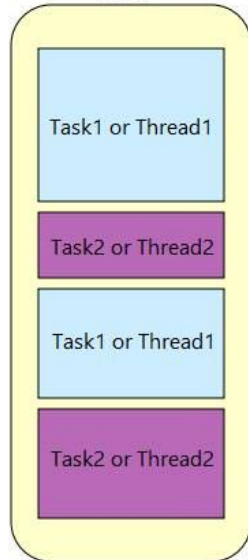
Concurrency

- THIẾT KẾ chương trình thành các tiến trình độc lập
- THIẾT KẾ để các tiến trình này có thể eventually parallel
- Đảm bảo rằng kết quả của chương trình không thay đổi so với chạy tuần tự

Concurrency & Parallelism

Concurrent

CPU

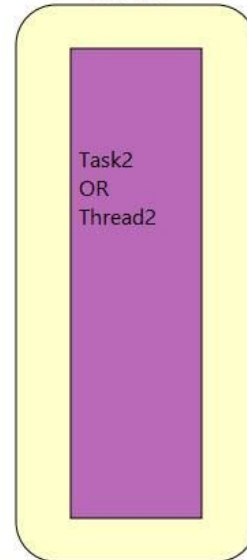


Concurrent [also] Parallel

CPU1



CPU2

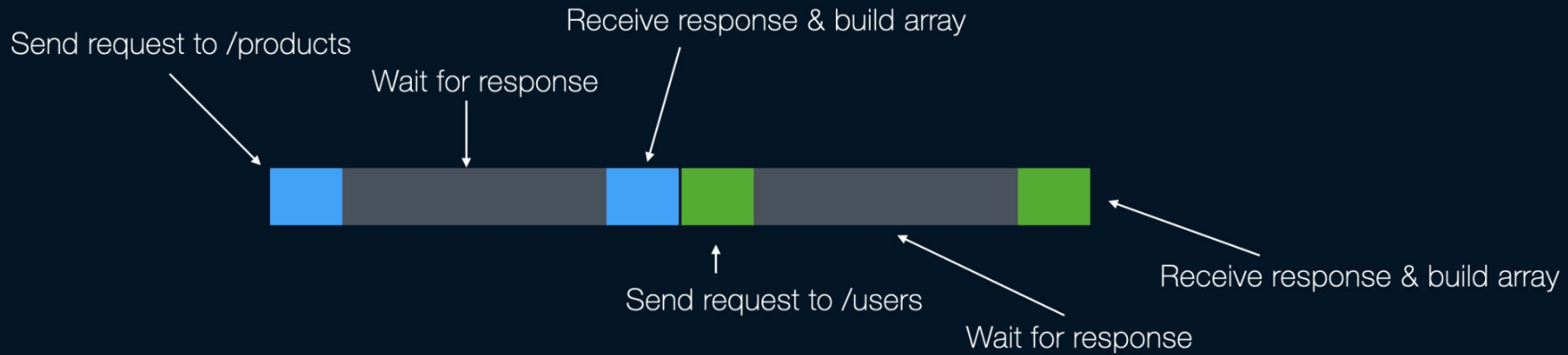




Tối ưu hiệu suất ứng dụng PHP với concurrency

Một ví dụ sync đơn giản

```
function names()  
{  
    $data = Http::get('data.location/products')->json();  
  
    $names = [];  
  
    foreach ($data as $item){  
        $names[] = $item['name'];  
    }  
  
    return $names;  
}
```





Concurrency trong thực tế

- Multiprocessing
- Multithreading
- Coroutine / Green thread / Fibers

Ý tưởng: Lưu giữ trạng thái của ứng dụng và switching context



Process

- Được quản lý bởi OS, chạy trên các core CPU
- Chạy cả parallel và concurrency
- Độc lập về resource tài nguyên (memory, disk)
- PHP được thiết kế theo nguyên tắc này



Switch to process 1

Start HTTP request to collect products

Switch to process 2

Start HTTP request to collect users

Switch to process 1

If a response came, then build products names array

Switch to process 2

If a response came, then build users names array



Thread

- Được quản lý và lập lịch bởi OS
- Concurrency (1 process có thể có nhiều thread)
- Share chung các tài nguyên giữa các thread
- Tiết kiệm tài nguyên hơn process do share chung
- Tài nguyên được phân phối bởi OS
- Thread safe



Coroutine

- Được quản lý và lập lịch bởi runtime
- Concurrency (1 thread có thể có nhiều routine)
- Được quản lý và lập lịch bởi runtime
- Tiêu tốn rất ít tài nguyên (so với thread).
- Runtime hoặc developer cần chủ động trong việc lập lịch và switching context giữa các routine



Concurrency with PHP



Coroutine with PHP Generator

- Generator cho phép context switching thông qua từ khóa yield

- [Generator::current](#) — Get the yielded value
- [Generator::getReturn](#) — Get the return value of a generator
- [Generator::key](#) — Get the yielded key
- [Generator::next](#) — Resume execution of the generator
- [Generator::rewind](#) — Rewind the iterator
- [Generator::send](#) — Send a value to the generator
- [Generator::throw](#) — Throw an exception into the generator
- [Generator::valid](#) — Check if the iterator has been closed
- [Generator::__wakeup](#) — Serialize callback



Coroutine with PHP Generator

```
<?php
function nums() {
    for ($i = 0; $i < 5; ++$i) {
        //get a value from the caller
        $cmd = (yield $i);

        if($cmd == 'stop')
            return;//exit the function
    }
}

$gen = nums();
foreach($gen as $v)
{
    if($v == 3)//we are satisfied
        $gen->send('stop');

    echo "{$v}\n";
}

//Output
0
1
2
3
?>
```

Spatie Fork

```
use Spatie\Fork\Fork;

$results = Fork::new()
    ->run(
        function () {
            sleep(1);

            return 'result from task 1';
        },
        function () {
            sleep(1);

            return 'result from task 2';
        },
        function () {
            sleep(1);

            return 'result from task 3';
        },
    );

// this code will be reached this point after 1 second
$results[0]; // contains 'result from task 1'
$results[1]; // contains 'result from task 2'
$results[2]; // contains 'result from task 3'
```



Coroutine with PHP Fiber

```
<?php
$fiber = new Fiber(function (): void {
    $value = Fiber::suspend('fiber');
    echo "Value used to resume fiber: ", $value, PHP_EOL;
});

$value = $fiber->start();

echo "Value from fiber suspending: ", $value, PHP_EOL;

$fiber->resume('test');
?>
```

The above example will output:

```
Value from fiber suspending: fiber
Value used to resume fiber: test
```



PHP Parallel extension

<https://www.php.net/manual/en/book.parallel.php>



Non-blocking IO

- IO là tác vụ cần kết nối đầu vào và đầu ra (ví dụ khởi tạo DB connect, gửi HTTP Request, kết nối Cache, Networking)
- Non-blocking: Chủ động nhường quyền switch context
- Coroutine yêu cầu các thư viện chủ động support non blocking IO

```
go(function(){  
  $data = Http::get('data.location/products')->json();  
  // yields  
  
  foreach(...)  
});
```

```
go(function(){  
  $data = Http::get('data.location/users')->json();  
  // yields  
  
  foreach(...)  
});
```



Các thư viện hỗ trợ Concurrency cho PHP

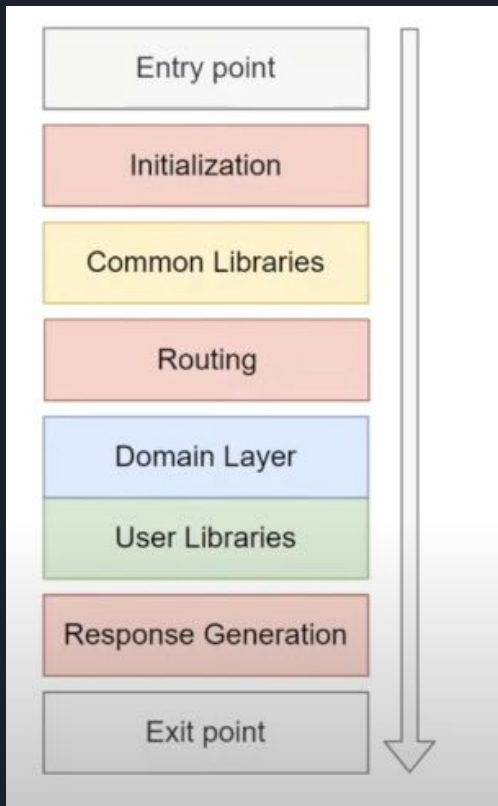
- React PHP: Xây dựng EventLoop
- Swoole: Xây dựng Coroutine
- Amps PHP: EventLoop, Promises, Coroutine,

=> Dễ dàng xây dựng kiến trúc hướng sự kiện (Event driven)



Laravel Octane

Cơ chế hoạt động của ứng dụng PHP truyền thống



- Một điểm vào (Entry point) (Mọi request sẽ đều bắt đầu vào index.php)
- Single threaded model
- Ứng dụng được bootstrap và terminate sau mỗi request

Ưu điểm:

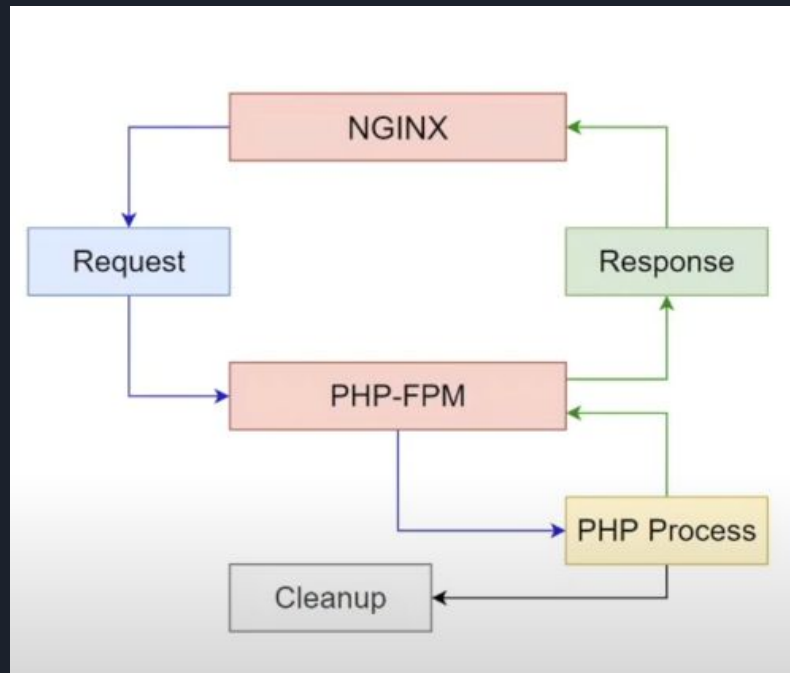
- Không memory leak, không race condition
- Memory được clean hoàn toàn giữa các request
- Dev đỡ đau đầu

Nhược điểm

- Cực kỳ tốn kém tài nguyên, nhất là CPU + tác vụ I/O
- Không tận dụng được sức mạnh của RAM
- Không Keep-Alive Connection
- Không Connection Pool

OPCache có thể hỗ trợ trợ load một số libraries, nhưng không triệt để.

Cơ chế hoạt động của ứng dụng PHP truyền thống





Laravel Octane

- Ứng dụng được bootstrap một lần duy nhất vào memory
- Tích hợp coroutine

=> Tối ưu tốc độ xử lý

FRANKENPHP

A modern app server for PHP



<https://frankenphp.dev>

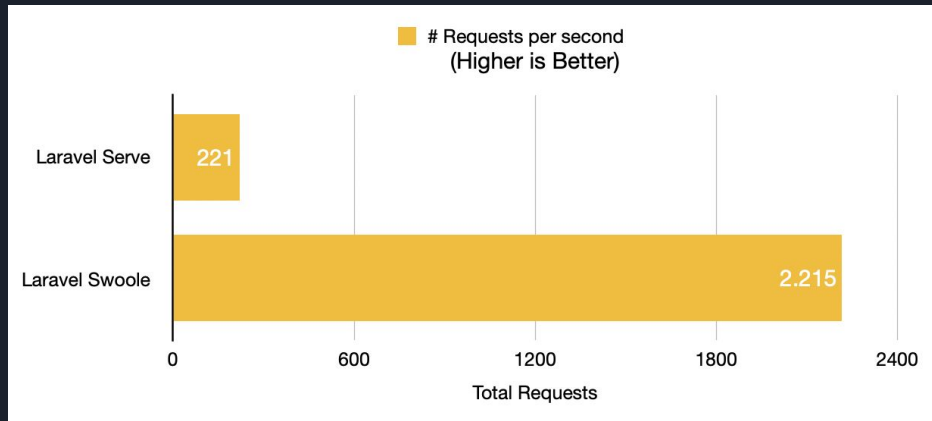
Web Application server

- Swoole
- RoadRunner
- Laravel Octane

Triển khai tích hợp dễ dàng (Docker)

Sử dụng được song song với Nginx

Các thử nghiệm đều cho thấy tốc độ tăng ít nhất 50 - 60% và giảm lượng connection tới DB





Thank you
Q&A